TCS Self Learning

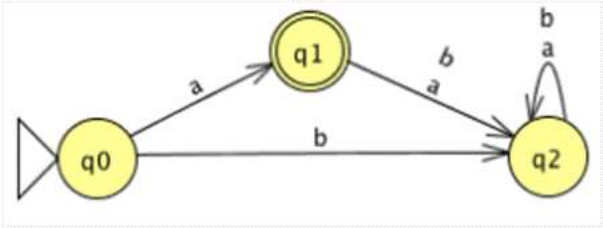
1) Deterministic Finite Automata

A DFA is a finite-state machine used to accept or reject finite strings of symbols. For every input string, there is exactly one possible computation path, so the machine always halts with a definite answer: accept or reject. If a DFA is denoted by M, then the language recognized by it is written as L(M), which is the set of all strings accepted by M.

Formal Definition:
A DFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

- Q is a finite set of states
- Σ is a finite set of input symbols (the alphabet)
- $\delta: Q \times \Sigma \rightarrow Q$ is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of accept states

Example: This DFA over the alphabet {a, b} accepts only the single string "a". Any other string is rejected.



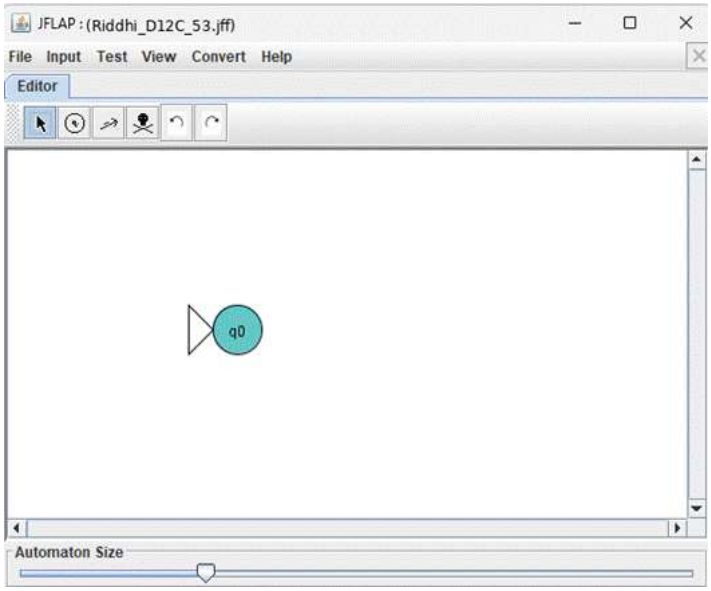
State	a	b
q_0	q_1	q_2
q_1	q_2	q_2
q_2	q_2	q_2

Each state has exactly one outgoing transition for every symbol in the alphabet. Since the next state is always uniquely fixed, the automaton is called deterministic.

Example:
Consider the regular language L over the alphabet { a, b, c } consisting of all strings that begin and end
with c. Let us construct a DFA M to recognize that language.
Some strings in the language are: c, cc, ccc, cac, cabc, cabcbabc
Some strings not in the language are: λ, a, ac, cb, ccca

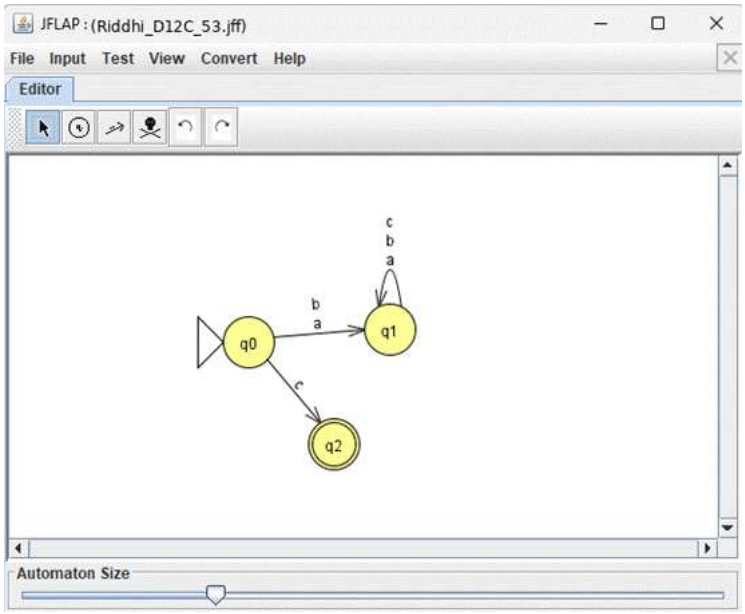
Execution:

Step 1: Create the initial state q0

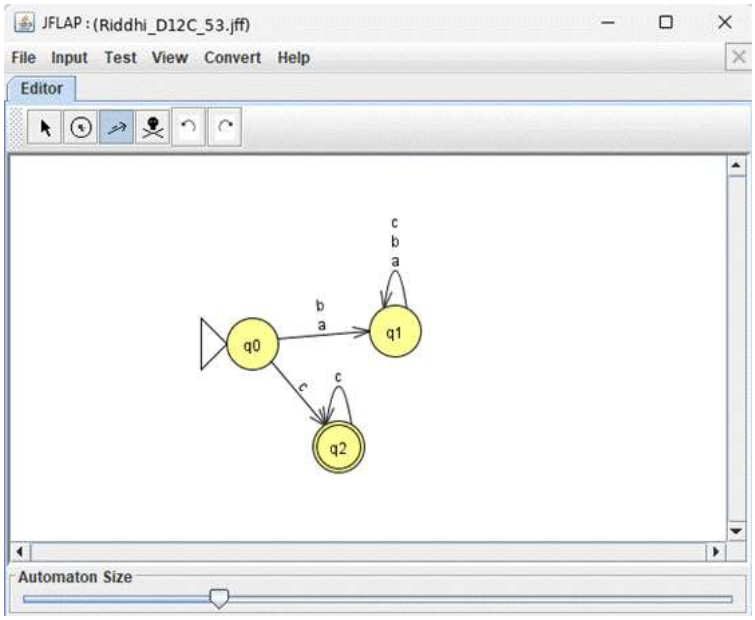


Step 2: Create a new state, q1, as the destination of a or b from q0 and which traps all subsequent
substrings.

Step 3: Create a new state, q2, as the destination of transition c from q0 and which is an accept state.

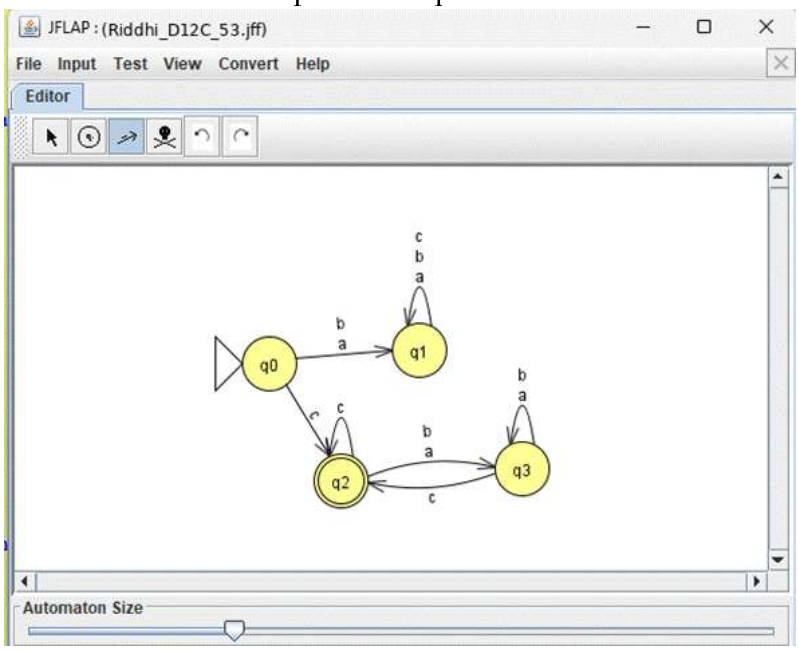


Step 4: Add a c transition to and from q2



Step 5: Add state q3 with a transition on a or b from q2. Add self-loop transitions on a or b so that the automaton remains in q3.

Add a transition from q3 back to q2 on c.



Testing with sample inputs:

Hence the DFA accepts all strings that start and end with c

2) Non Deterministic Finite Automata

An NFA is also a finite-state machine that accepts or rejects finite strings of symbols, but unlike a DFA, its computation for a given input does not have to be unique. From one state on one input symbol, it may move to multiple next states or to none at all. An NFA N recognizes the language L(N), which contains all strings for which at least one computation path finishes in an accept state.

Example: This NFA over {a, b} recognizes all strings that end in "ab". In q_0 it keeps reading and, whenever it sees an a, can guess that this a is the second-to-last symbol and move to q_1. It then needs a b to reach q_2, and any further symbol kills that path.

Formal Definition

An NFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:

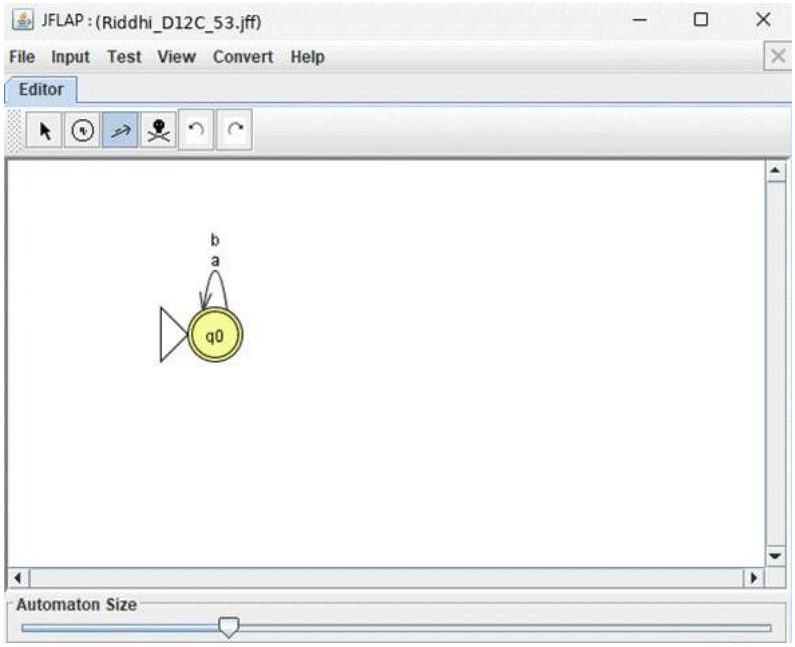
- Q is a finite set of states
- Σ is a finite set of input symbols (the alphabet)
- $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow P(Q)$ is the transition function, which returns a set of states (possibly empty)
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of accept states

Example:

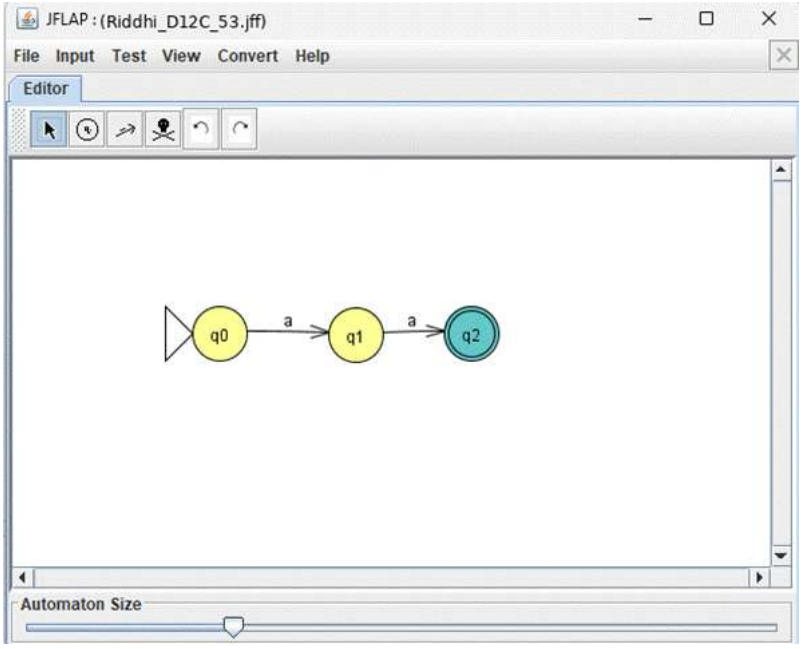
Define an NFA that recognizes the following language L over $\Sigma = \{a, b\}$:
$L = \{ w \mid w \text{ ends with aa} \}$.

Execution:

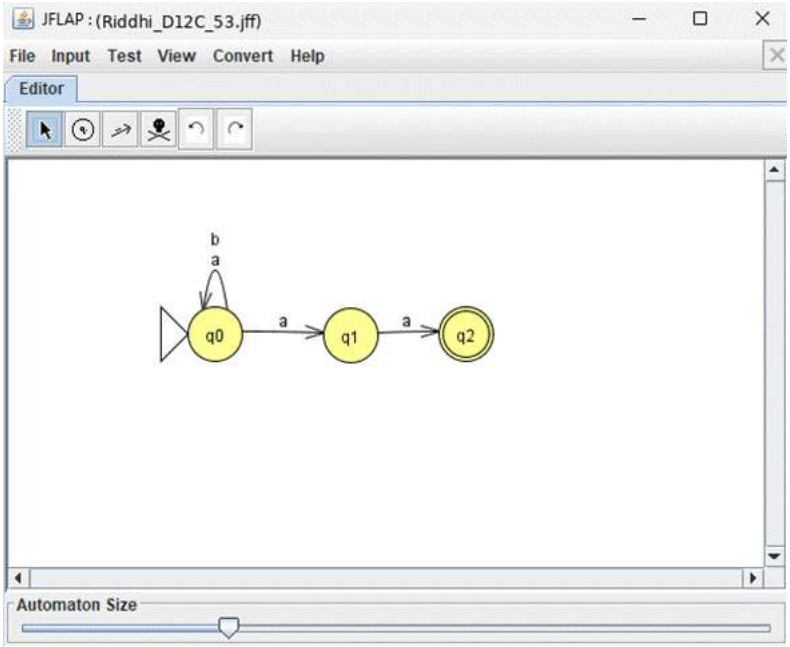
Step 1: Construct an NFA that recognizes the set of all strings {a, b}*



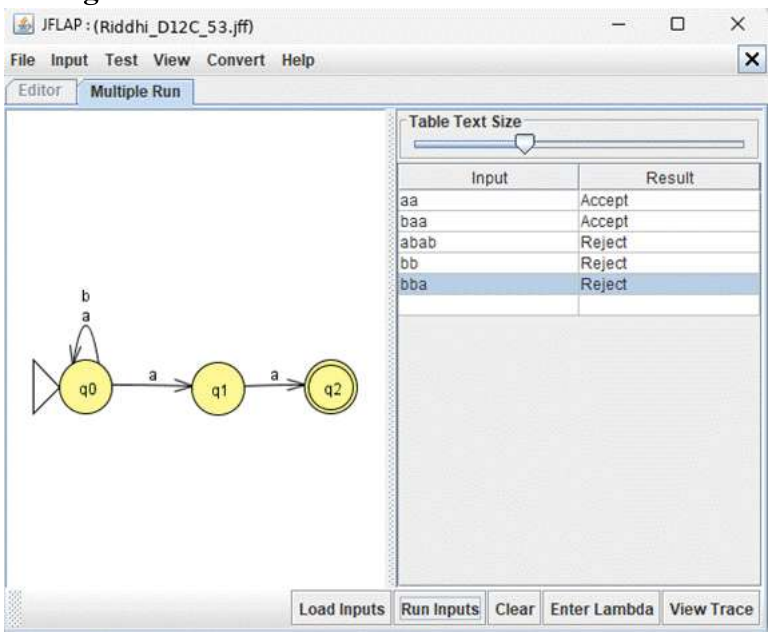
Step 2: Construct an NFA that recognizes the string aa, as shown below



Step 3: Create an NFA that recognizes the sequence of these strings (their concatenation) by combining the final state of the first NFA with the initial state of the second NFA.



Testing:



Hence, this NFA successfully accepts all strings ending with aa.

3) NFA to DFA Conversion

For every NFA, there exists an equivalent DFA that accepts the same language. The subset construction method forms the DFA by treating each DFA state as a set of NFA states, thereby tracking all states that the NFA could possibly occupy after reading the input processed so far.

Construction

Given an NFA $N = (Q, \Sigma, \delta, q_0, F)$, build a DFA $D = (Q', \Sigma, \delta', q_0', F')$, where:

- $Q' = P(Q)$: each DFA state is a subset of NFA states
- $q_0' = \{q_0\}$: the start state is the set containing only the NFA's start state
- $\delta'(S, a) = \bigcup \delta(q, a)$ for $q \in S$: on symbol a, move to the union of all states reachable from any state in S
- $F' = \{S \in Q' \mid S \cap F \neq \emptyset\}$: a DFA state accepts if it contains at least one NFA accept state

If the NFA has ε-transitions, take the ε-closure (all states reachable by ε-moves alone) of the start state and of every result of δ'.

Procedure

1. Start with the DFA state $\{q_0\}$ (or its ε-closure).
2. For each new DFA state S and each symbol a, compute $\delta'(S, a)$ as the union of the NFA transitions.
3. If the result is a new subset, add it as a state.
4. Repeat until no new subsets appear.
5. Mark every subset containing an NFA accept state as accepting.

Only subsets reachable from the start are needed, so the DFA is usually much smaller than the full 2^n.

Example

Convert the NFA from before (strings over {a, b} ending in "ab", start q_0, accept q_2):

State	a	b
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\emptyset$	$\emptyset$

Starting from $\{q_0\}$:

- $\{q_0\}$ on a $\rightarrow \{q_0, q_1\}$; on b $\rightarrow \{q_0\}$
- $\{q_0, q_1\}$ on a $\rightarrow \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$; on b $\rightarrow \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$
- $\{q_0, q_2\}$ on a $\rightarrow \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$; on b $\rightarrow \{q_0\} \cup \emptyset = \{q_0\}$

No new subsets appear, so the DFA has three states. Naming A = $\{q_0\}$, B = $\{q_0, q_1\}$, and C = $\{q_0, q_2\}$, the DFA is $(\{A, B, C\}, \{a, b\}, \delta', A, \{C\})$, with δ' given by:

State	a	b
A	B	A
B	B	C
C	B	A

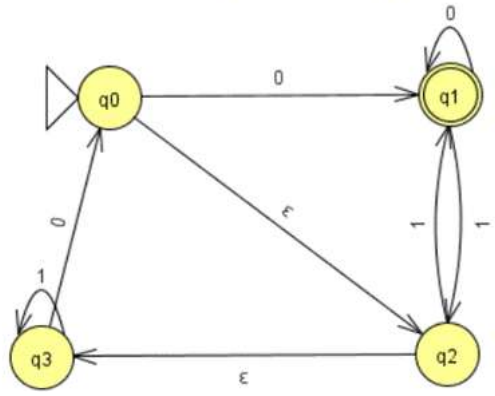
C is the only accepting state, since it is the only subset containing q_2. The other five subsets ($\emptyset$, $\{q_1\}$, $\{q_2\}$, $\{q_1, q_2\}$, $\{q_0, q_1, q_2\}$) are unreachable, so they are left out.

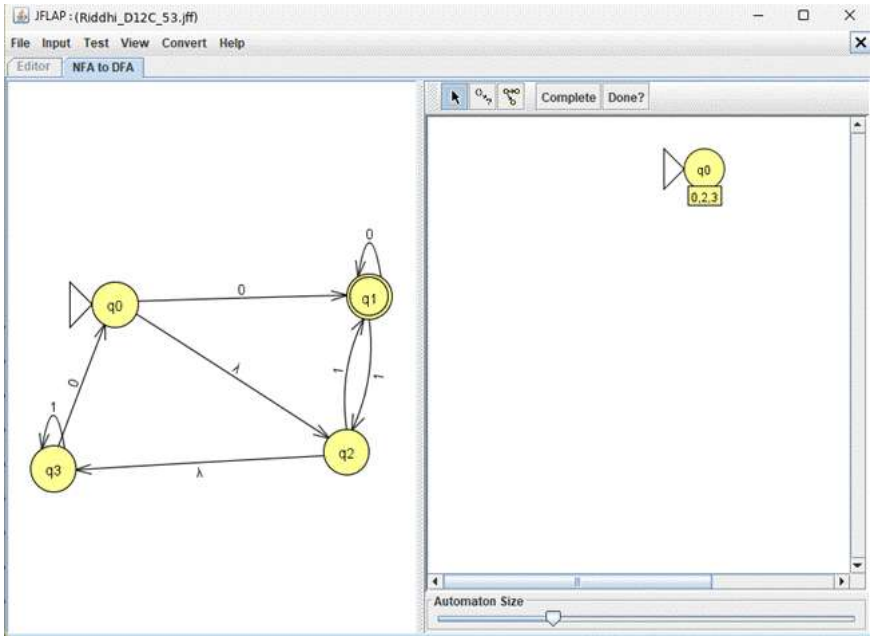
Key points

- The construction proves NFAs and DFAs recognize exactly the same languages, the regular languages.
- The DFA can need up to 2^n states for an n-state NFA, and some languages really do require this blowup.
- The empty set $\emptyset$ acts as the DFA's "dead" state when it is reachable.

Example:

Convert the following NFA to an equivalent DFA

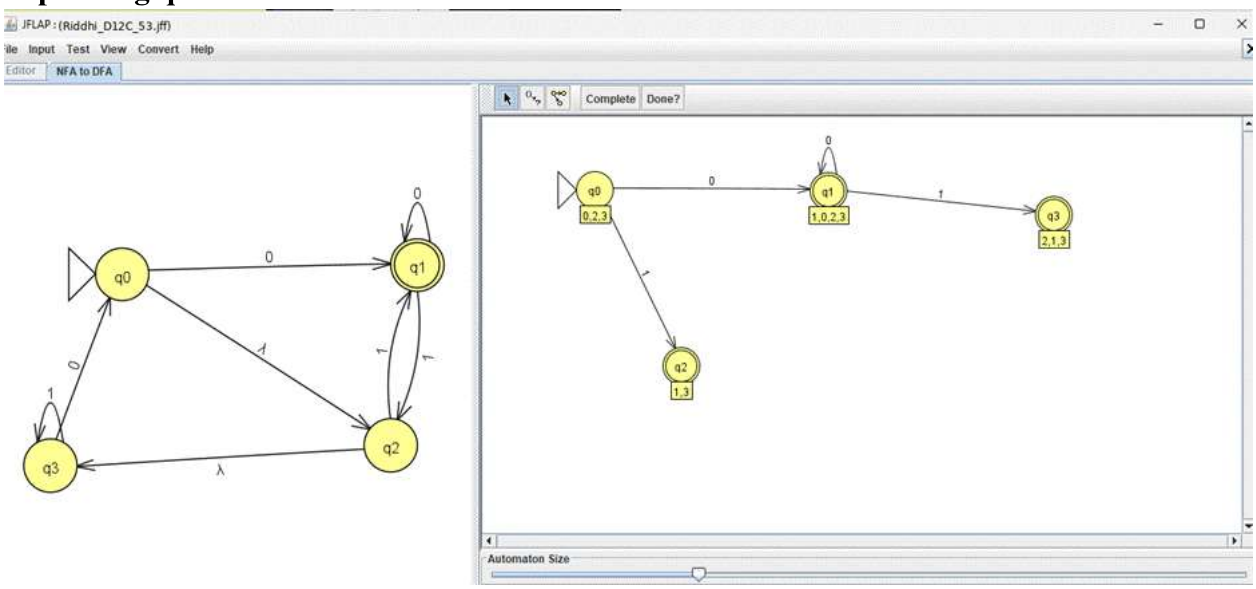




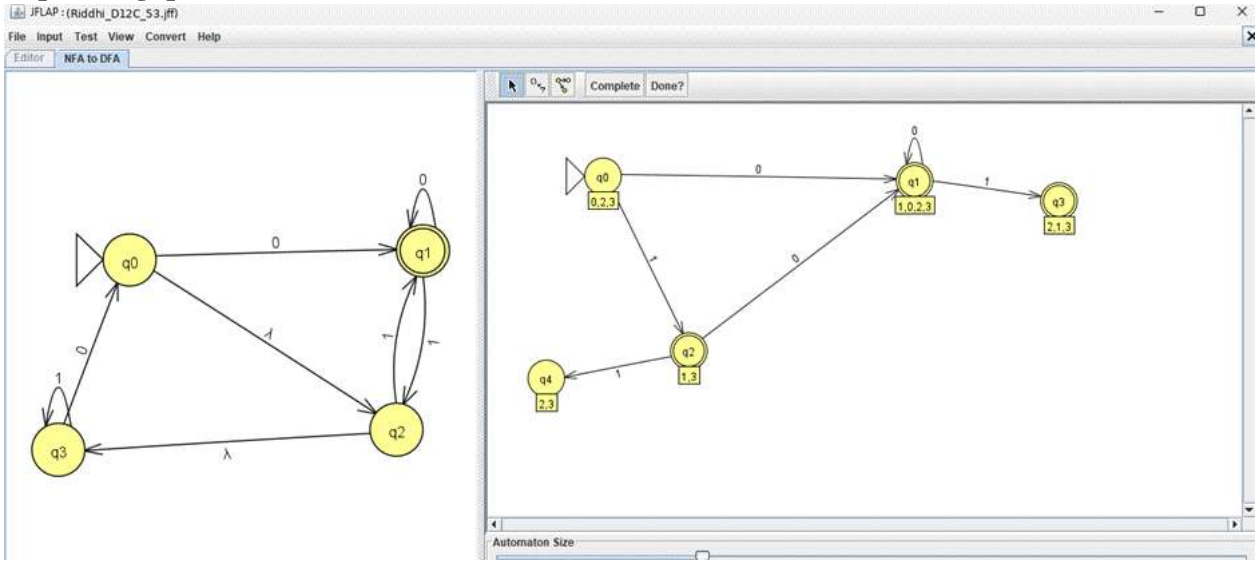
Step 1: In the NFA, q0 goes to q1. q2 goes nowhere on a 0. And finally q3 goes to q0 on a 0. So currently we have the subset of states {q0, q1}. But due to transitions the state will be {q0, q1, q2, q3}
ick the state expander button, then click the state that you want to expand.
Doing this will create 2 transitions, one for 0 and one for 1.

Step 2: Repeat the process of clicking the state expander button and then clicking the states. Do this for each of the states

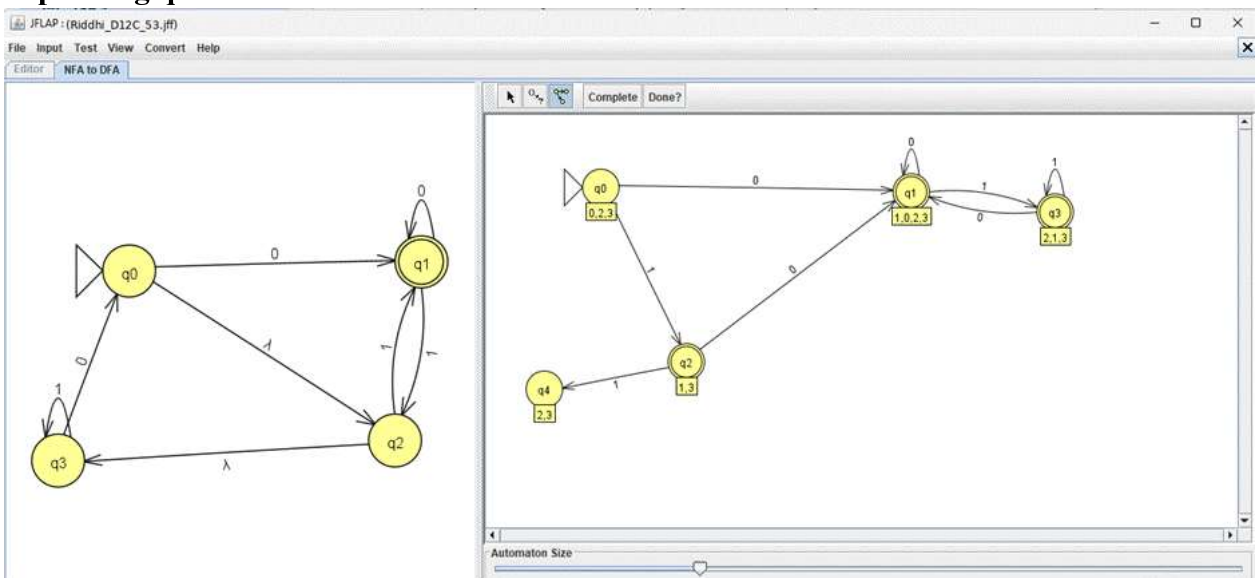
Expanding q1



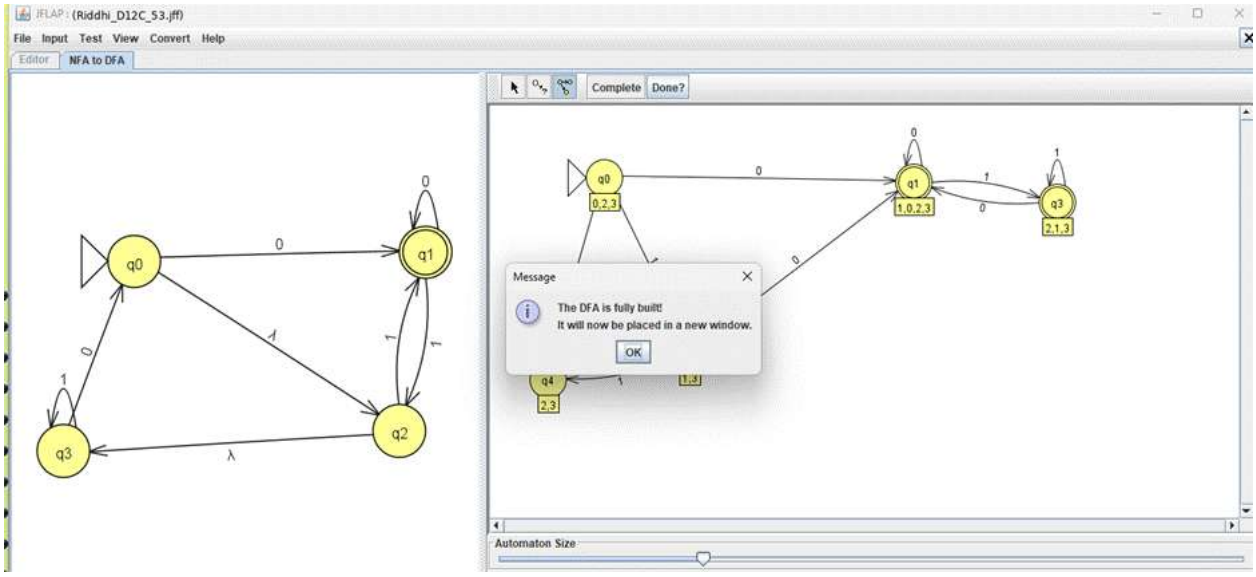
Expanding q2



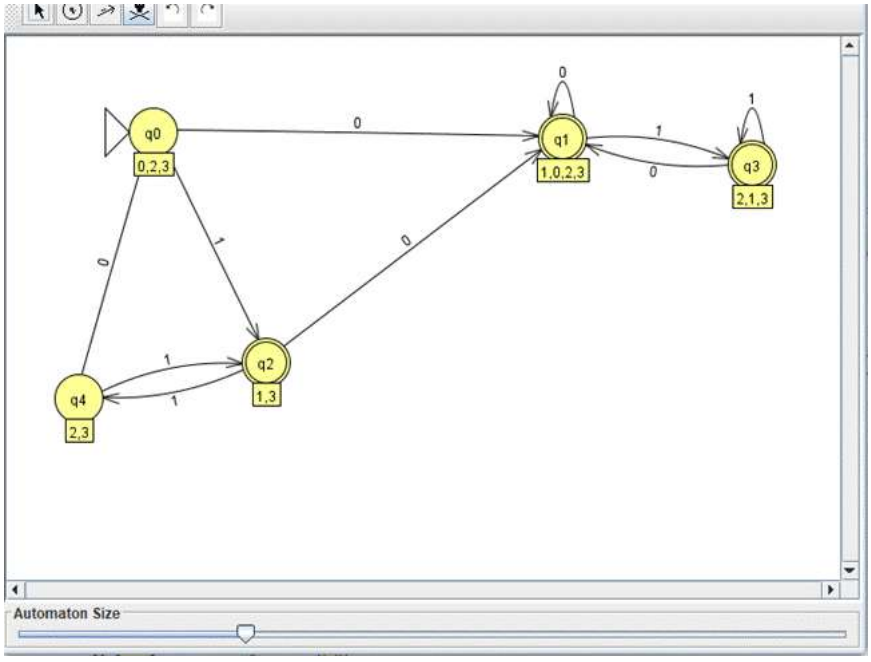
Expanding q3



Expanding q4



Final DFA



4) DFA Minimization

Different DFAs may recognize the same language, but each regular language has one unique minimal DFA, up to state renaming. Minimization reduces the DFA by removing unreachable states and merging states that are behaviorally equivalent.

Key Idea

Two states p and q are equivalent if, for every string w, starting from p and starting from q lead to the same accept/reject outcome. Equivalent states can be merged without changing the language. If some string w gives different outcomes, the states are distinguishable.

Procedure (partition refinement)

1. Remove unreachable states. Delete any state that can't be reached from q_0.

2. Split by acceptance. Start with two groups: accepting states F and non-accepting states Q \ F.
3. Refine. Split a group if two of its states send some symbol a to different groups. Repeat until no group splits.
4. Merge. Each final group becomes one state of the minimal DFA. The start state is the group containing q_0, the accept states are the groups inside F, and transitions follow the group of the target.

Equivalently, the table-filling method marks pairs of states as distinguishable, starting with (accepting, non-accepting) pairs, then marking any pair that moves to an already marked pair on some symbol. Unmarked pairs at the end are equivalent.

Example

Take this DFA over {a, b} with start state 0 and F = {2, 3}:

State	a	b
0	1	2
1	1	3
2	1	3
3	1	2
4	0	4

- Step 1: State 4 is unreachable, so remove it.
- Step 2: The groups are {0, 1} (non-accepting) and {2, 3} (accepting).
- Step 3: In {0, 1}, on a both states go to 1 (same group), and on b they go to 2 and 3 (same group). In {2, 3}, on a both go to 1, and on b they go to 3 and 2 (same group). Nothing splits, so the partition is final.
- Step 4: Merge into P = {0, 1} and Q = {2, 3}.

The minimal DFA is ({P, Q}, {a, b}, δ, P, {Q}), with δ given by:

State	a	b
P	P	Q
Q	P	Q

It recognizes the strings that end in b. Five states reduced to two.

Already minimal

The 3-state DFA from the NFA conversion (strings ending in “ab”) is already minimal. The initial groups are {A, B} and {C}. On b, A goes to A but B goes to C, so {A, B} splits, leaving three singleton groups.

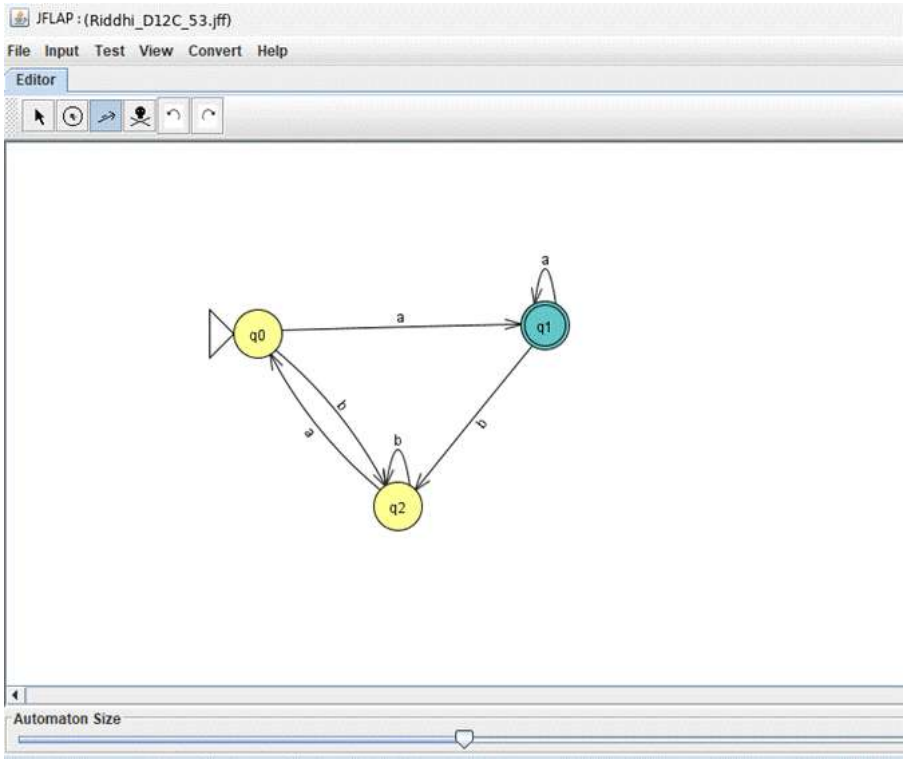
Key points

- Minimization never changes the language, only the number of states.
- The minimal DFA is unique, which gives a way to test whether two DFAs are equivalent: minimize both and compare.
- This relies on the Myhill-Nerode theorem: the number of states in the minimal DFA equals the number of distinguishable classes of strings for the language.

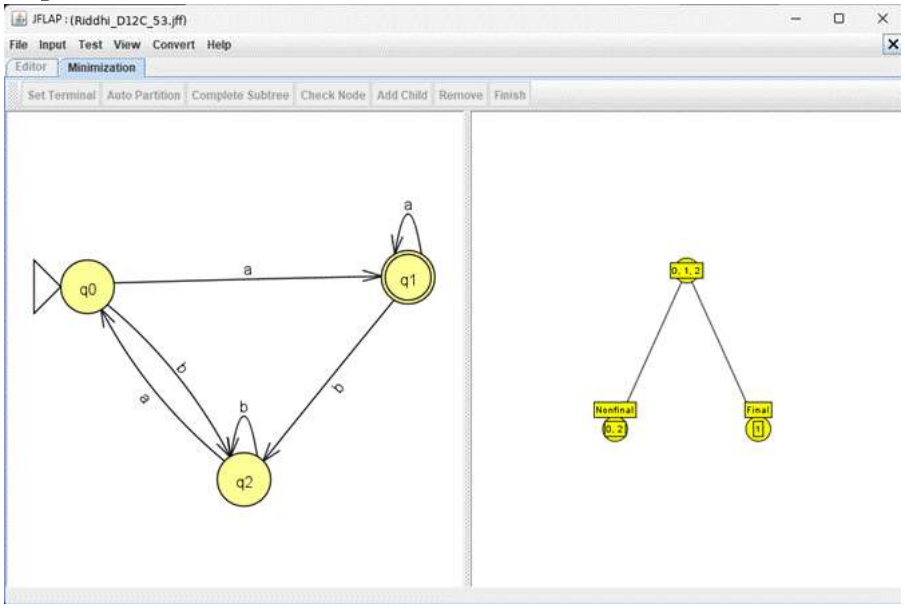
- Hopcroft's algorithm does the refinement in O(n log n) time for n states.

Example:
Let us consider the language L1 = {w ∈ Σ∗|w is λ or w ends in a}.

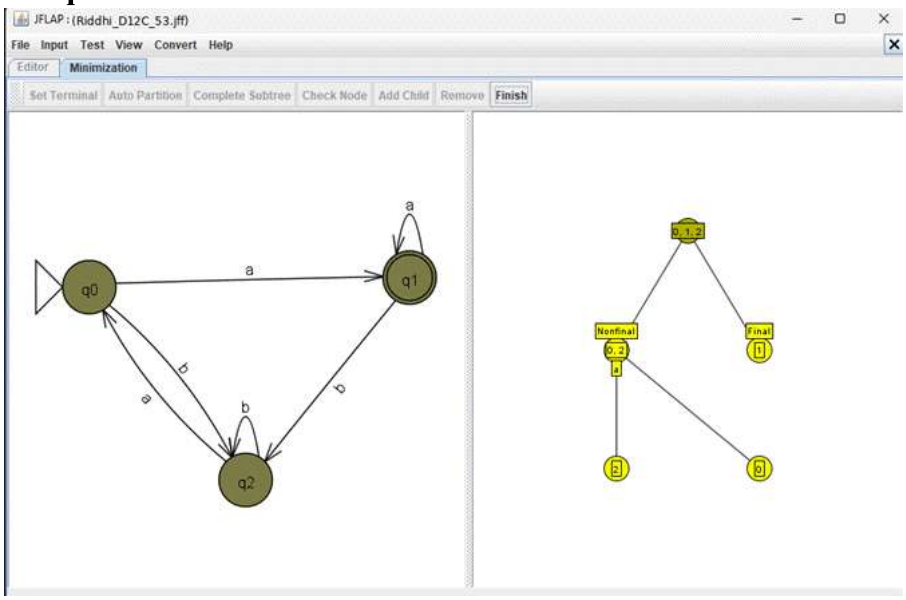
Execution:
DFA:



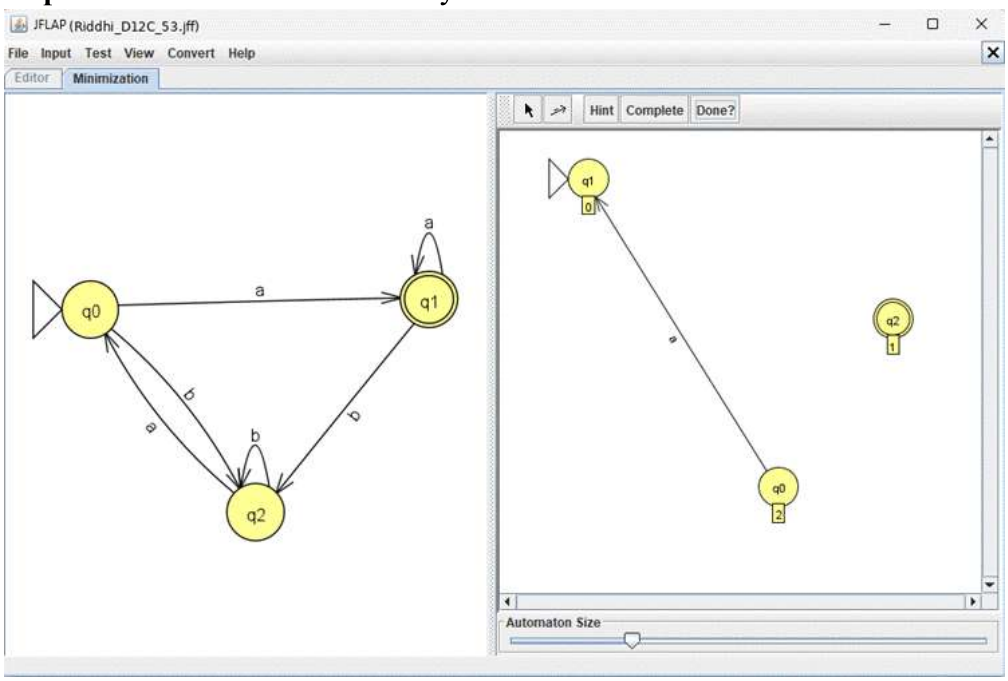
Step 1:

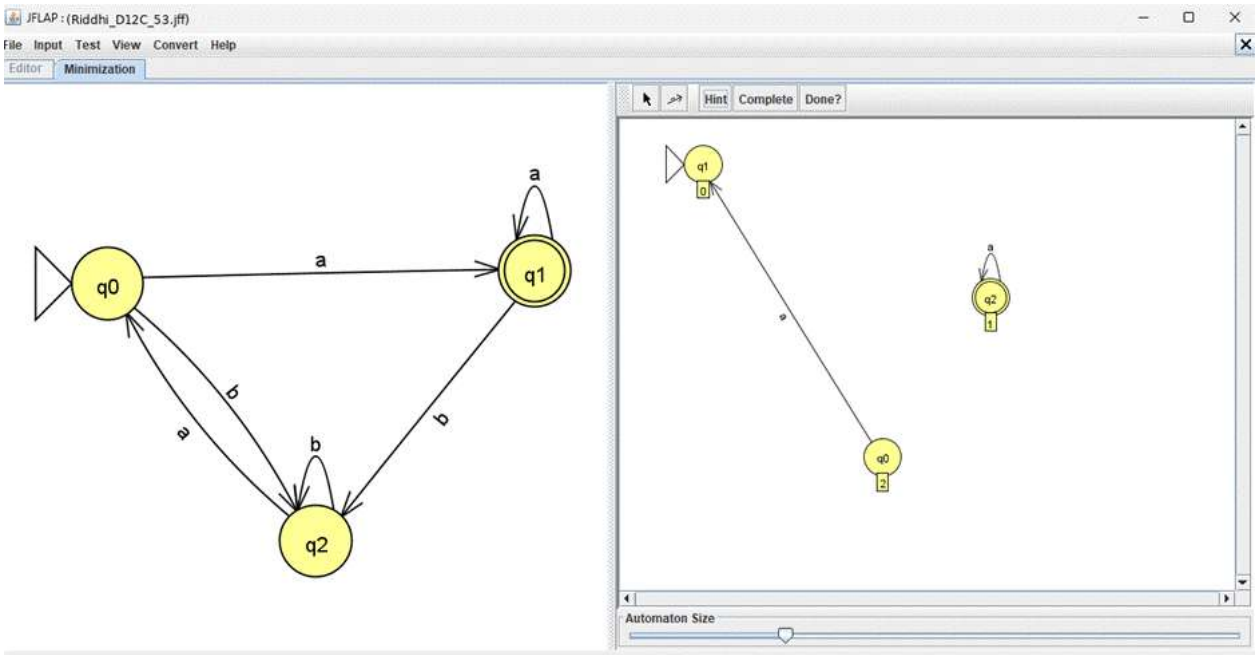


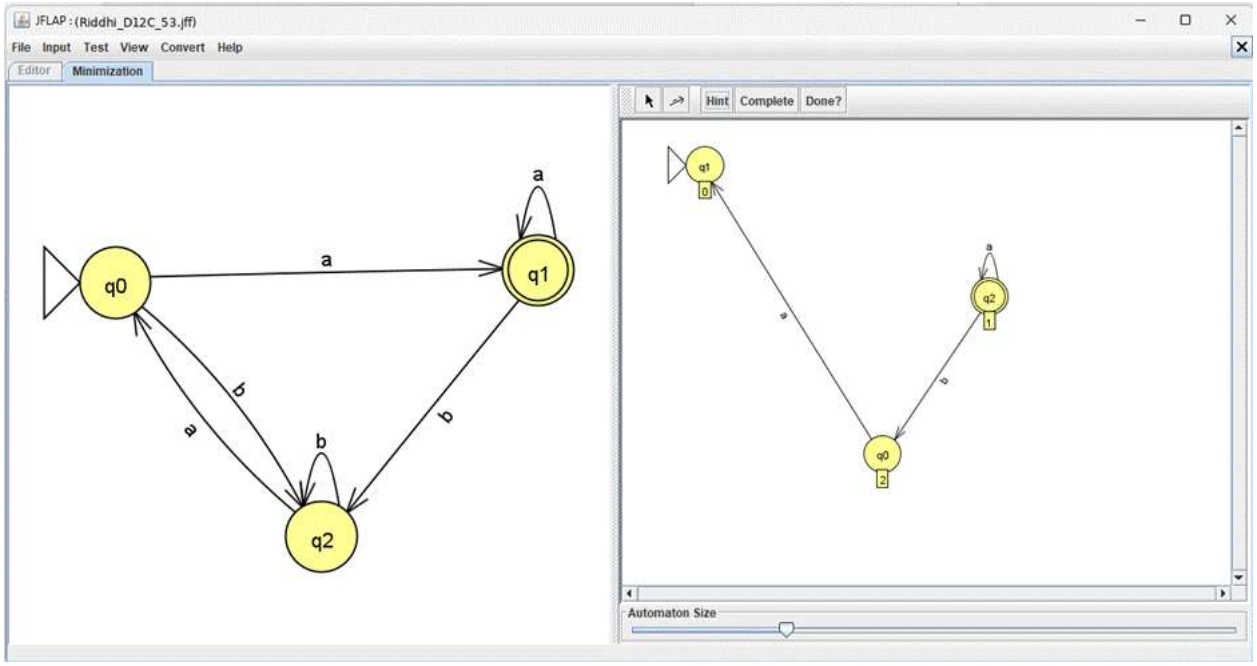
Complete Subtree

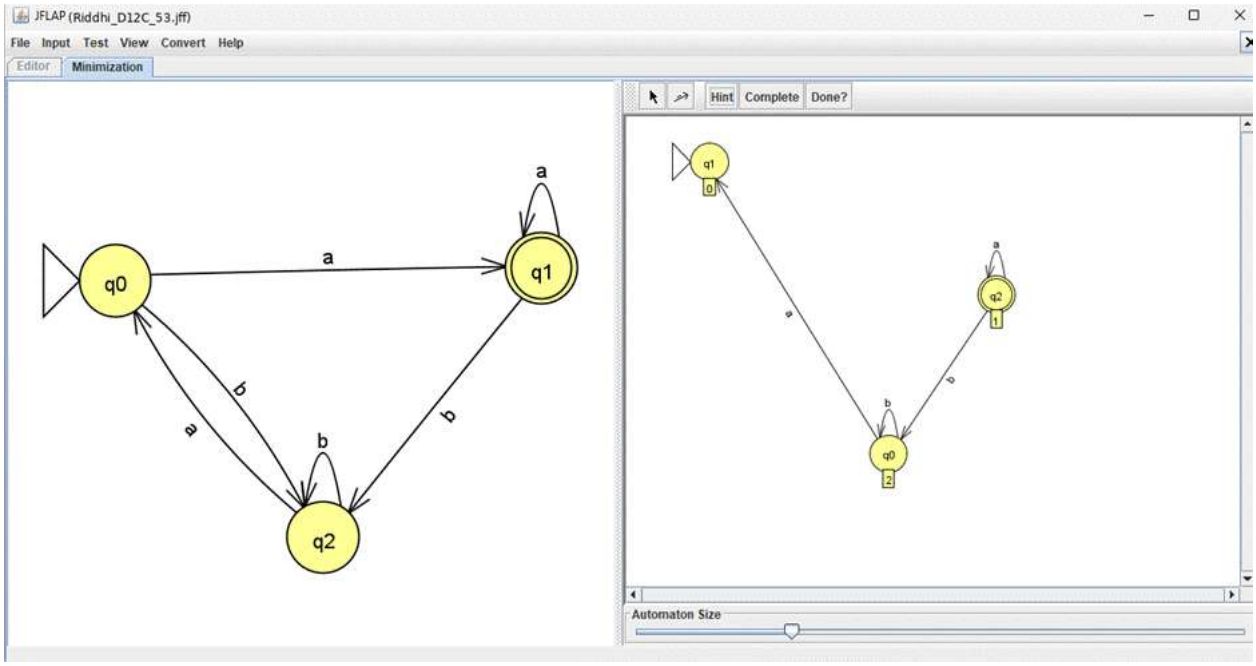


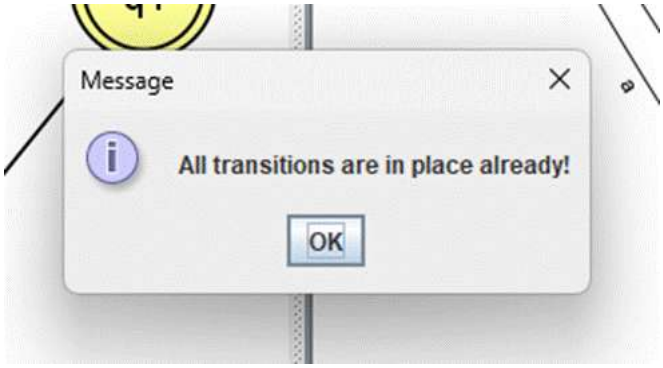
Step 2: Place the transitions one by one



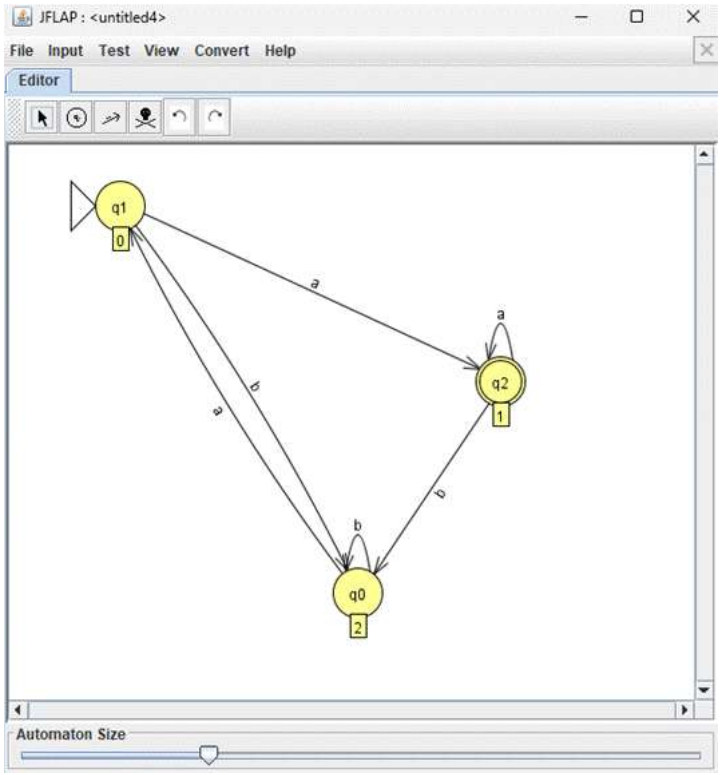

JFLAP : (Riddhi_D12C_53.jff)
File Input Test View Convert Help
Editor Minimization
Hint Complete Done?
q0
q1
q2
a
b
Automaton Size


JFLAP : (Riddhi_D12C_53.jff)
File Input Test View Convert Help
Editor Minimization
Hint Complete Done?
q0
q1
q2
a
b
Automaton Size


JFLAP (Riddhi_D12C_53.jff)
File Input Test View Convert Help
Editor Minimization
Hint Complete Done?
q0
q1
q2
a
b
Automaton Size


Message
All transitions are in place already!
OK

Minimized DFA:



5) DFA to Regular Grammar Conversion

Any DFA can be transformed into an equivalent regular grammar that generates exactly the same language. The basic idea is to convert each state into a non-terminal and each transition into a production rule so that the derivation follows the same behavior as the DFA run.

Regular Grammars

A **right-linear grammar** has productions of the form A → aB, A → a, or A → ε, where A and B are non-terminals and a is a terminal. Each rule produces at most one terminal followed by at most one non-terminal, at the right end. Right-linear grammars generate exactly the regular languages.

Construction

Given a DFA D = (Q, Σ, δ, q_0, F), build a grammar G = (V, Σ, P, S), where:

- **V = Q**: each state becomes a non-terminal
- **S = q_0**: the start state becomes the start symbol
- **Transition rule:** for each δ(p, a) = q, add the production **p → aq**
- **Accept rule:** for each p ∈ F, add the production **p → ε**

Reading a symbol corresponds to producing a terminal and moving to the next non-terminal. Reaching an accept state allows the derivation to end with ε.

Procedure

1. Name a non-terminal for each state, with the start state as the start symbol.
2. For every transition p —a→ q, write p → aq.
3. For every accept state p, write p → ε.

4. Optionally, drop any non-terminal that can never lead to a string (a dead or trap state) along with the productions that use it.

Example

Take the DFA from the NFA conversion (strings over {a, b} ending in "ab"), with start state A and accept state C:

State	a	b
A	B	A
B	B	C
C	B	A

The grammar has V = {A, B, C}, start symbol A, and these productions:

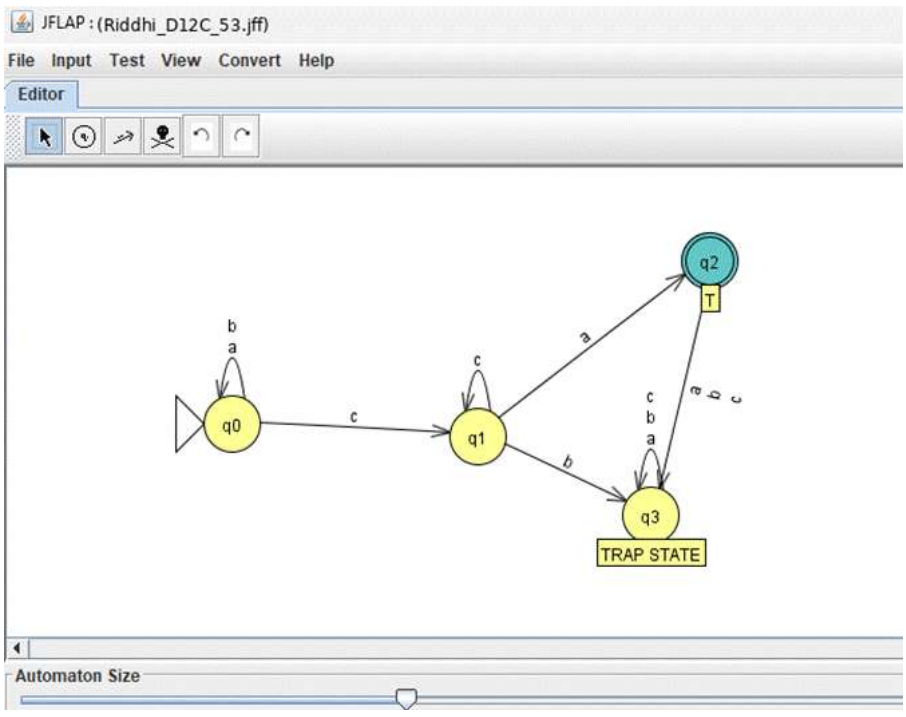
- A → aB | bA
- B → aB | bC
- C → aB | bA | ε

Derivation of "aab": A ⇒ aB ⇒ aaB ⇒ aabC ⇒ aab

This matches the DFA's run on "aab": A →a→ B →a→ B →b→ C, which ends in the accept state C. A string like "aba" fails: the derivation reaches C after "ab", but then the only way to continue is a → B, which cannot end in ε.

Example:

Let us consider this DFA



State q0 labeled R we have the following (→ is pronounced reduces to)

R → cS

R → aR

R → bR

S → aT

S → cS
T → null
Hence DFA has been converted to Grammar

6) DFA to Regular Expression Conversion

A DFA can also be converted into an equivalent regular expression that describes exactly the language accepted by the automaton. A standard technique for this is state elimination, where states are removed one by one and the transition labels are replaced with suitable regular expressions until a single final expression remains.

Key Idea

Allow edges to be labeled with regular expressions instead of single symbols. This is called a generalized NFA (GNFA). When a state is removed, every path that went through it is replaced by a direct edge whose label is the regular expression for that detour. The language does not change.

Procedure

1. Add a new start state S with an ε-edge to the old start state.
2. Add a new accept state F with ε-edges from every old accept state. These old states are no longer accepting.
3. Merge parallel edges. If several edges go from p to q, replace them with one edge labeled by the union ($R_1 \mid R_2 \mid \ldots$). If there is no edge, treat it as ∅.
4. Eliminate each original state q one at a time. For every pair of states p and r with an edge into q and an edge out of q, add or extend the edge p → r with the label R_pq (R_qq)* R_qr:
    - R_pq is the label from p to q
    - R_qq is the label of the loop on q (use ε if none)
    - R_qr is the label from q to r
    - If p → r already exists, union the new label with the old one.
5. Stop when only S and F remain. The label on the edge S → F is the regular expression.

Example

Take the DFA from the earlier sections (strings over {a, b} ending in "ab"), with start state A and accept state C:

State	a	b
A	B	A
B	B	C
C	B	A

After adding S and F, the edges are:

- S → A: ε
- A → A: b, A → B: a
- B → B: a, B → C: b
- C → B: a, C → A: b, C → F: ε

Eliminate A (loop b, so the middle term is b*):

- S → B: ε · b* · a = b*a

- C → B: a | (b · b* · a) = a | b^{+}a = b*a

Eliminate C (no loop; the only edge in is from B on b, and the edges out are to B on b*a and to F on ε):

- B → B: a | (b · b*a) = b*a
- B → F: b

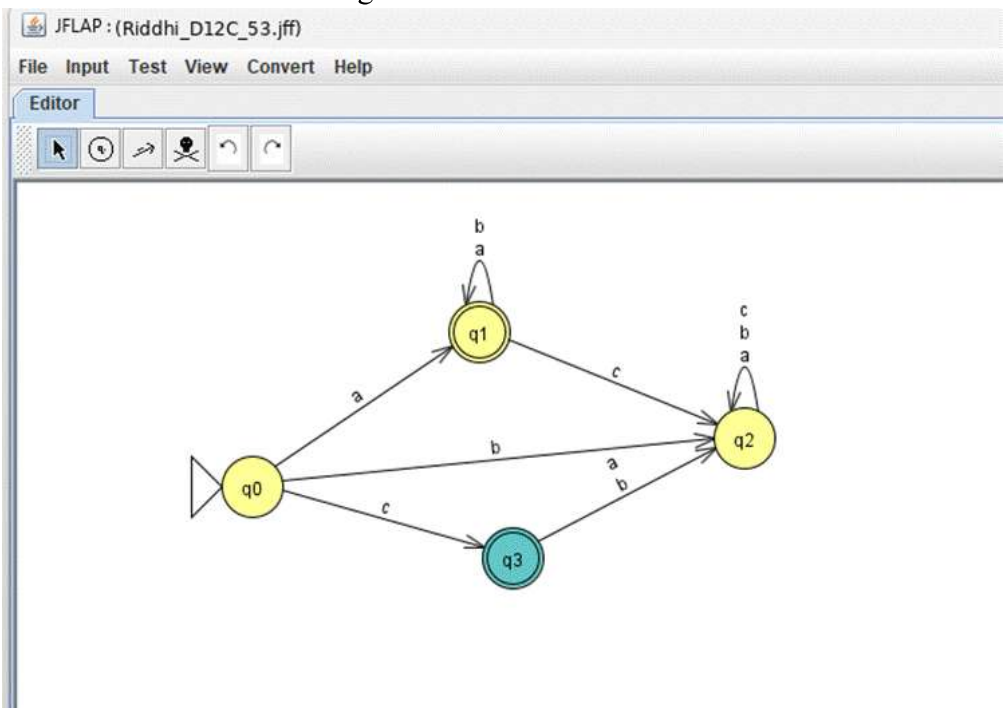
Eliminate B (loop b*a, so the middle term is (b*a)*):

- S → F: b*a · (b*a)* · b = $(b*a)^{+}$b

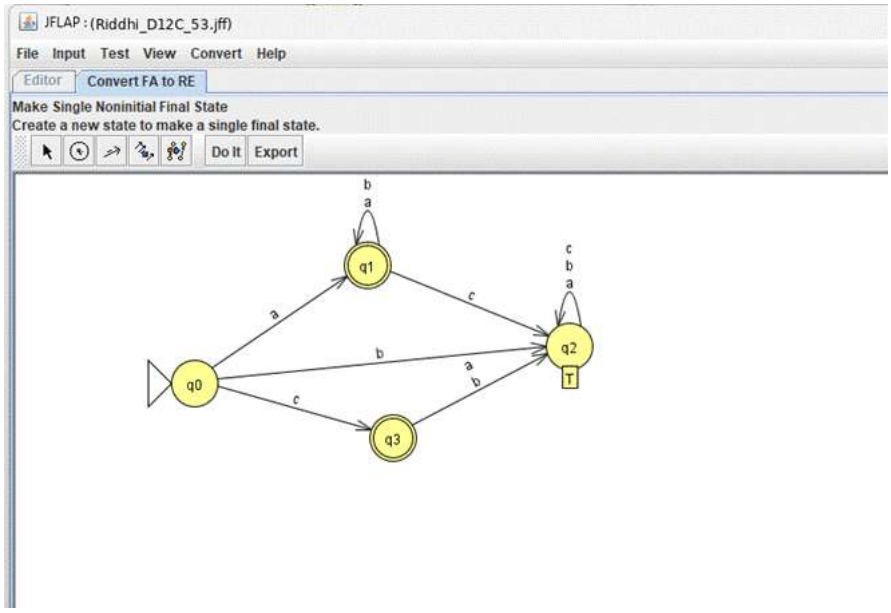
The regular expression is $(b*a)^{+}$b. It is equivalent to the more familiar (a|b)*ab, since a string ends in "ab" exactly when it is a string ending in a (that is, $(b*a)^{+}$) followed by b.

Example

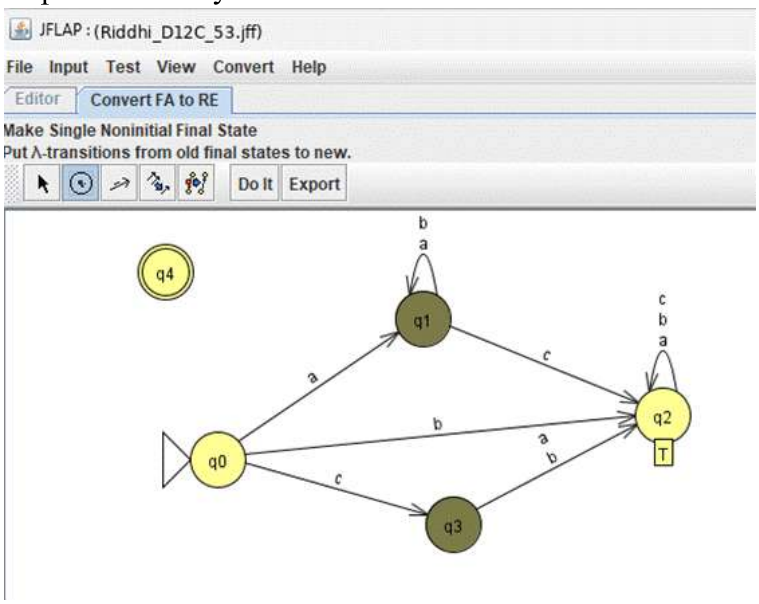
Let us consider the following DFA



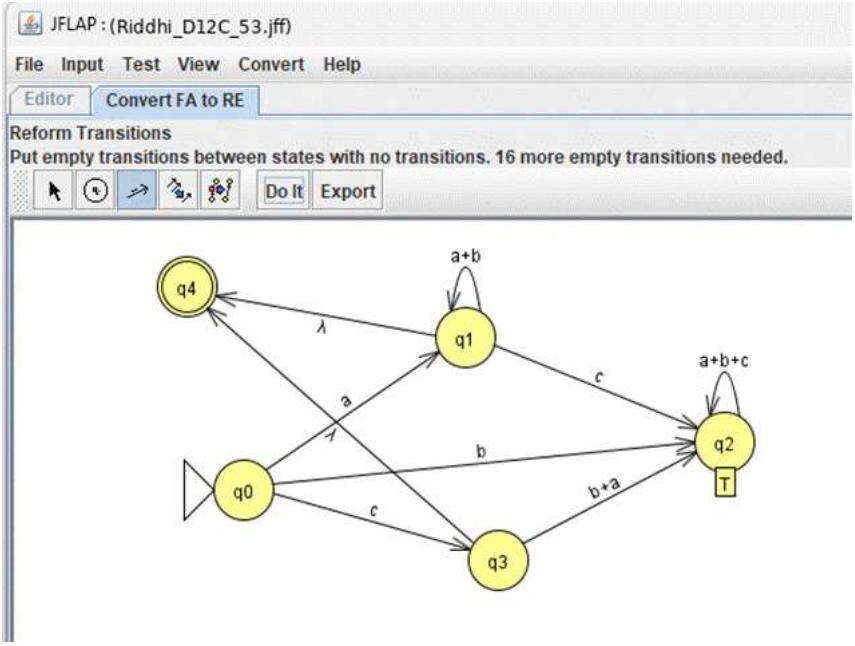
Step 1:



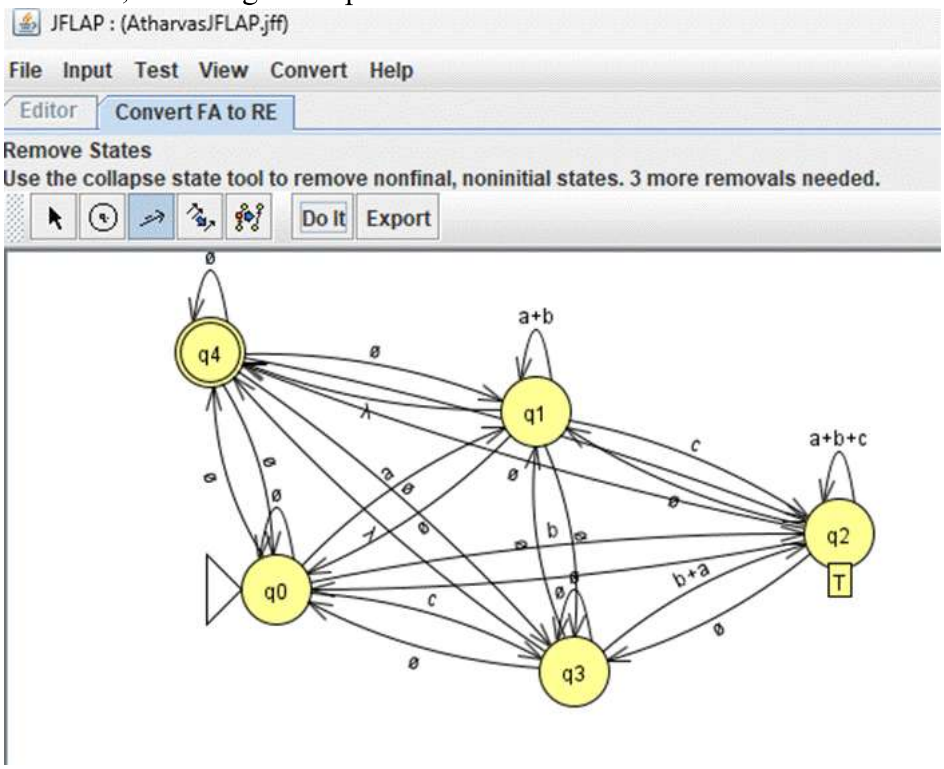
Step 2: Manually create a new state.



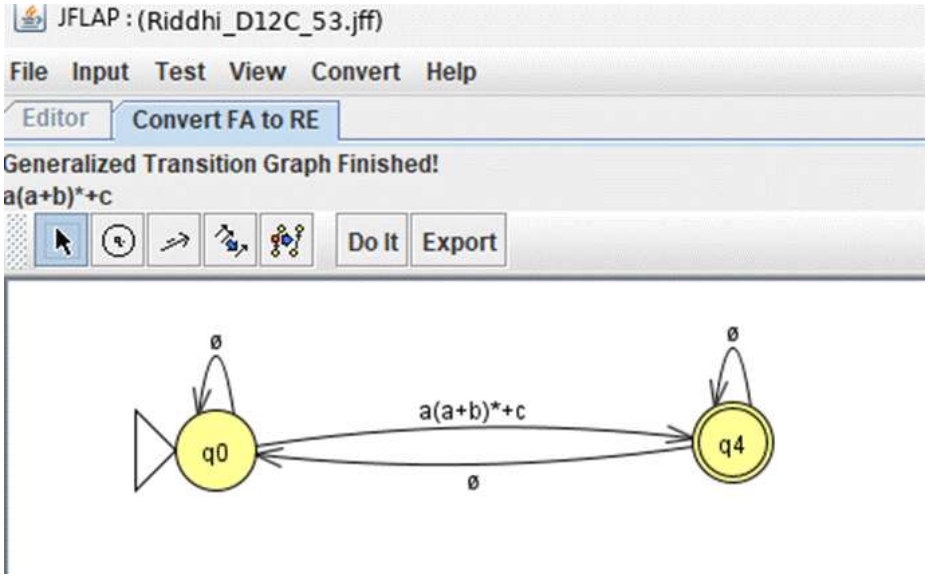
Step 4: Next, collapse parallel transitions, labeling the resulting transitions with the equivalent regular expressions. Here is the result.



Step 5: Add empty transitions between every pair of states that lack an explicit existing transition, including states paired with themselves.



Step 6: Finally, use the collapse-state tool to eliminate states by replacing them with transitions labeled by the equivalent regular expression. The final result is shown below.



Hence we have successfully converted DFA to RE

7) Combining Automata

Regular languages are closed under operations such as union, intersection, difference, complement, concatenation, and Kleene star. Corresponding automata constructions exist for each of these operations.

Product construction (union, intersection, difference)

Given DFAs $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ over the same alphabet, run both in parallel:

- States: $Q_1 \times Q_2$
- Start: (q_1, q_2)
- Transition: $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$
- Accept states:
  - Union: $p \in F_1$ or $q \in F_2$
  - Intersection: $p \in F_1$ and $q \in F_2$
  - Difference $L_1 \setminus L_2$: $p \in F_1$ and $q \notin F_2$

Only accept set changes between operations.

Complement

Swap accepting and non-accepting states: $F' = Q \setminus F$. The DFA must be complete (a transition on every symbol from every state).

ε-NFA constructions

- Union: add a new start state with ε-edges to both start states.
- Concatenation L_1L_2: add ε-edges from each accept state of M_1 to the start state of M_2. Only F_2 accepts.
- Star L^*: add a new accepting start state with an ε-edge to the old start state, and ε-edges from each old accept state back to the old start state.

Example

Let M_1 accept an even number of a's (states E, O) and M_2 accept strings ending in b (states N, Y). The product starts at (E, N):

State	a	b
(E, N)	(O, N)	(E, Y)
(E, Y)	(O, N)	(E, Y)
(O, N)	(E, N)	(O, Y)
(O, Y)	(E, N)	(O, Y)

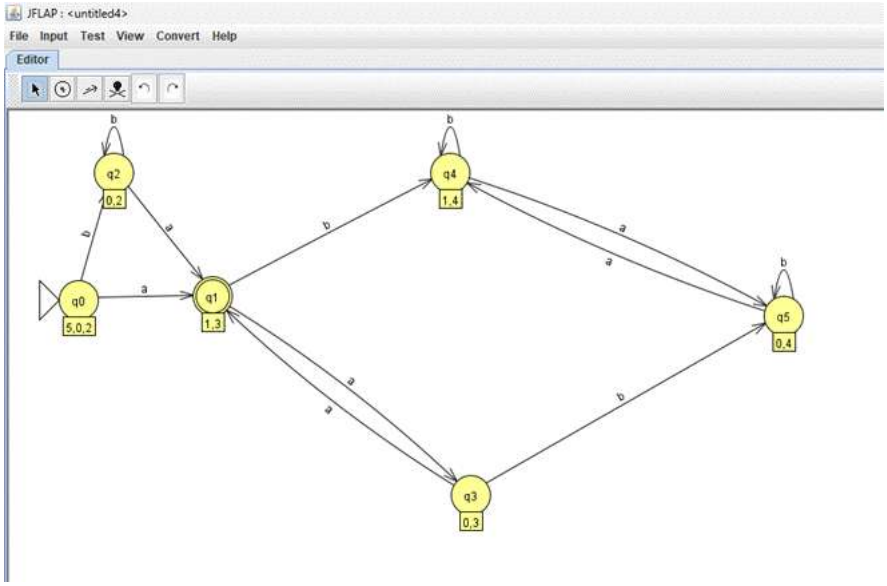
- Intersection: accept {(E, Y)}
- Union: accept {(E, N), (E, Y), (O, Y)}
- Difference $L_1 \setminus L_2$: accept {(E, N)}

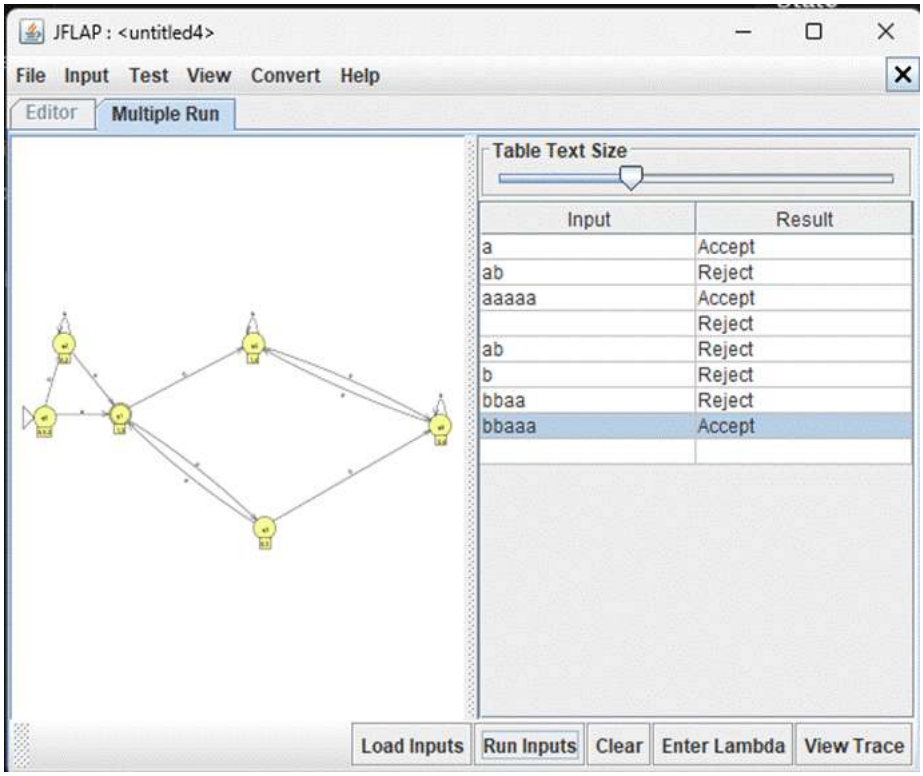
Example:

In this Exercise, we will use the "Combine Automata" feature of JFLAP to find the intersection of two DFA. For the alphabet $\Sigma = \{a, b\}$, consider the languages

L1 = {w ∈ Σ∗|w has an odd number of a's}, and

L2 = {w ∈ Σ∗|w does not have ab as a substring}





Hence this DFA accepts all strings that have an odd number of a's and have no ab substring

8) Regular Expression to DFA conversion

Regular Expression to DFA Conversion

First construct an ε-NFA from the given regular expression using Thompson's construction, and then convert that ε-NFA into a DFA using subset construction. Minimization may be applied afterwards if required.

Step 1: Thompson's construction

Each piece has one start state and one accept state:

- **a (a symbol):** start —a→ accept
- **R | S:** a new start with ε-edges to the starts of R and S, and ε-edges from their accept states to a new accept state
- **RS:** an ε-edge from R's accept state to S's start
- **R*:** a new start and a new accept state. The new start has ε-edges to R's start and to the new accept. R's accept has ε-edges back to R's start and to the new accept.

Step 2: Subset construction

1. The start state is the **ε-closure** of the NFA's start state (all states reachable by ε-moves alone).
2. For each DFA state S and symbol a, compute **ε-closure(move(S, a))**.
3. Repeat until no new sets appear.
4. A set is accepted if it contains the NFA's accept state.

Example: (a|b)*ab

The Thompson NFA gives four DFA states:

- **A** = ε-closure of the start state

- **B** = the state reached after reading an a (the NFA is now ready to read b)
- **C** = the state reached after reading “ab” (contains the NFA accept state)
- **D** = the state reached after reading a b from A, B-less (same behavior as A)

State	a	b
A	B	D
B	B	C
C	B	D
D	B	D

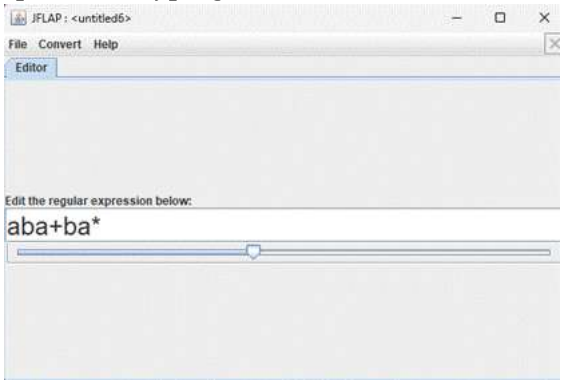
C is the only accepting state. A and D have identical rows, so minimization merges them:

State	a	b
A	B	A
B	B	C
C	B	A

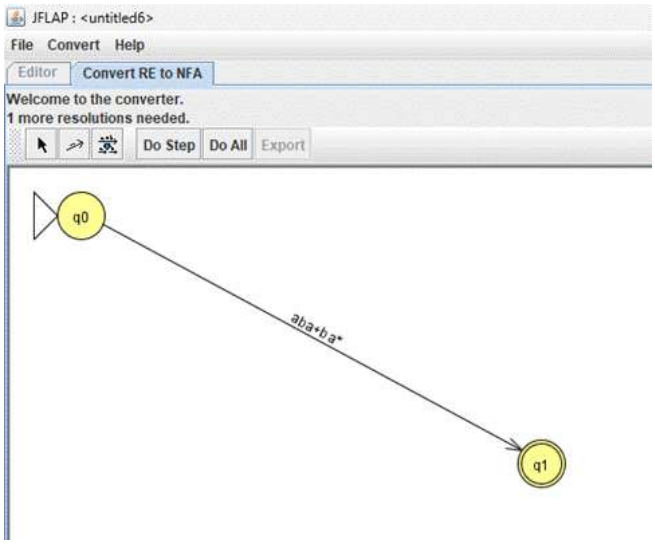
This is the 3-state DFA for strings ending in “ab” from the earlier sections.

Example: Convert the following Regular Expression to a DFA aba ∪ ba∗

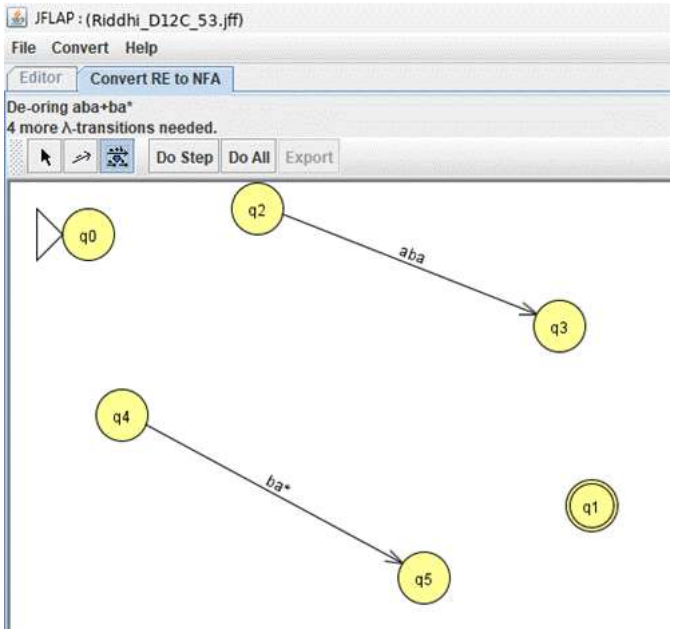
Step 1: The regular expression can be entered into JFLAP by choosing the regular expression option and typing in aba + ba∗, since JFLAP uses + to represent the ∪.



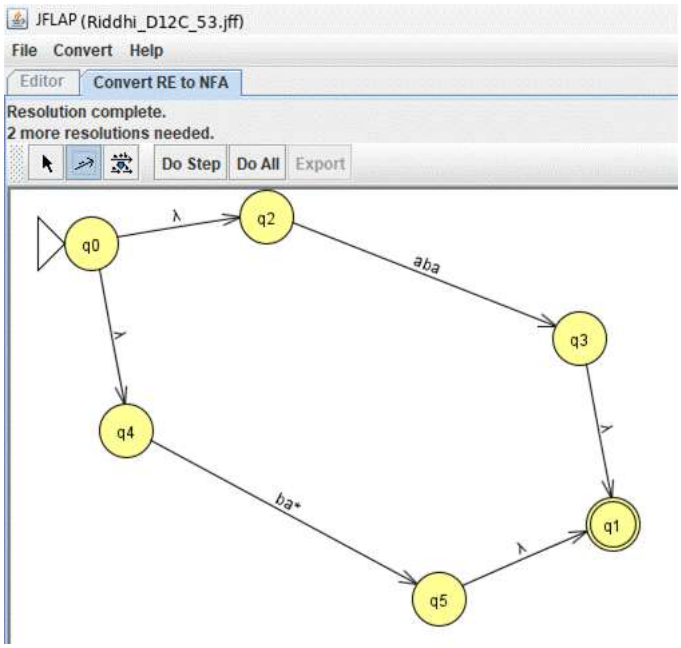
Step 2: Convert the regular expression to an NFA. This is done by breaking the regular expression down into its constituent parts. If you click the convert menu and choose ’Convert to NFA’ you will see the following.



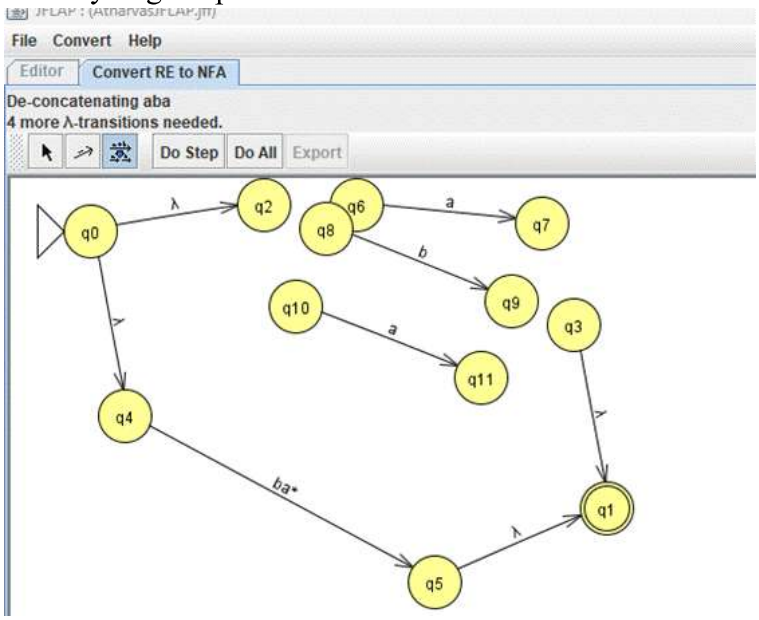
Step 3: Click the 'de-expressionify' transition button and then click the transition to get the two regular expressions that are separated by the union operator. So you get one transition of aba and one of ba



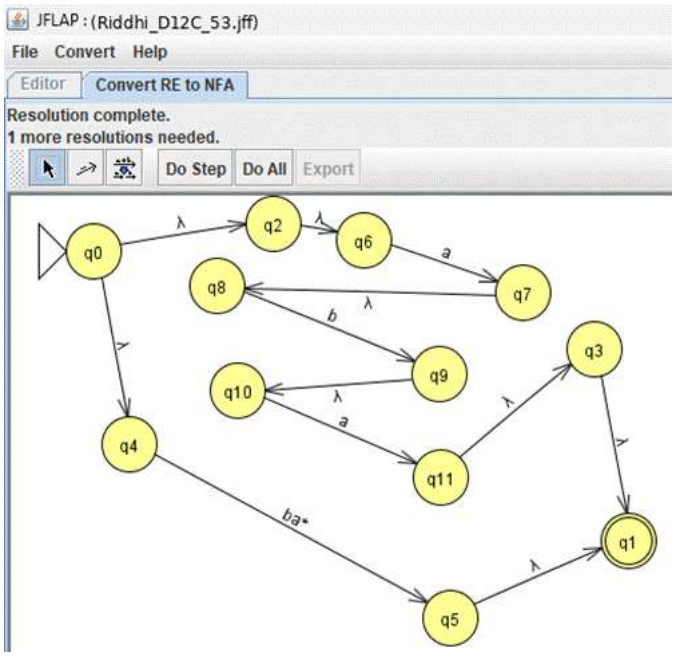
Step 4: Now to connect up these 'pieces' of the regular expression we will use epsilon transitions.



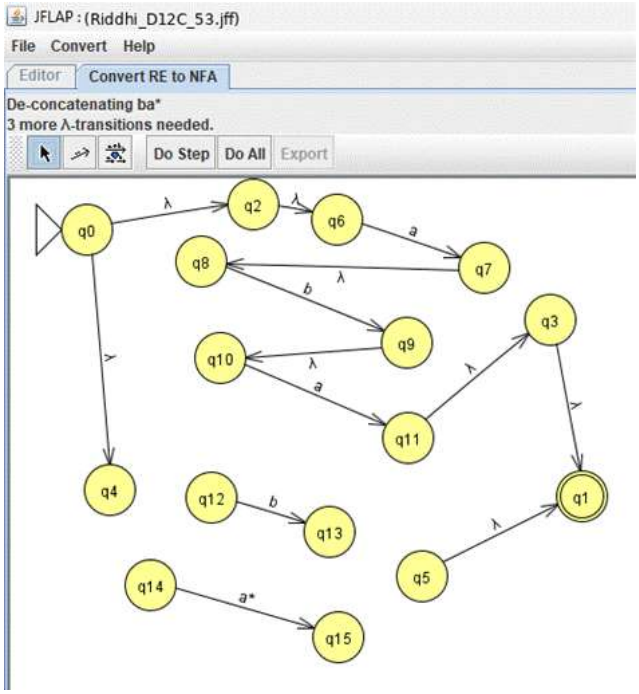
Step 5: In the next step we choose to break the expression aba. This is basically an a being concatenated with a b being concatenated with an a. Upon clicking the De-expressionify button you get a picture that looks like.



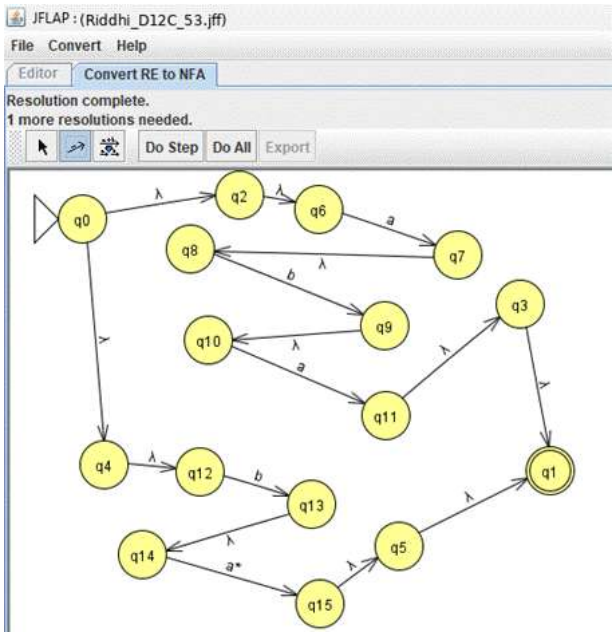
Step 6: Connect up the states via epsilon transitions. For concatenation you just need to transition
from the final state of one to the start state of the next via an epsilon as can be seen here



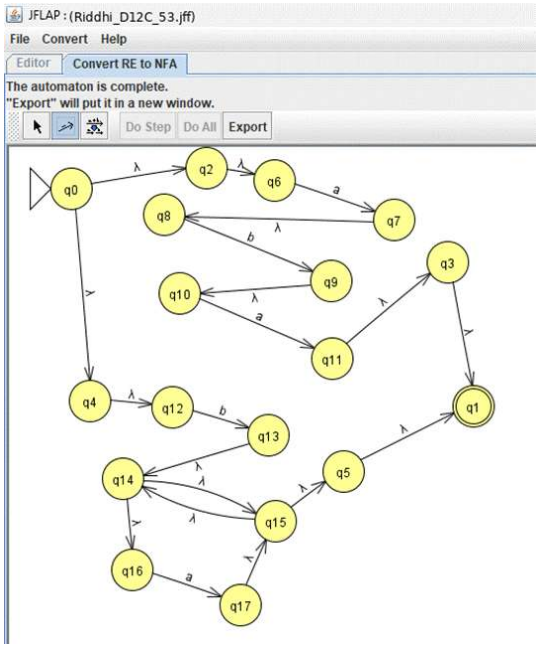
Step 7: Once aba has been broken up into its constituent NFA pieces, we turn our attention to ba∗ which upon de-expressionfication first looks like



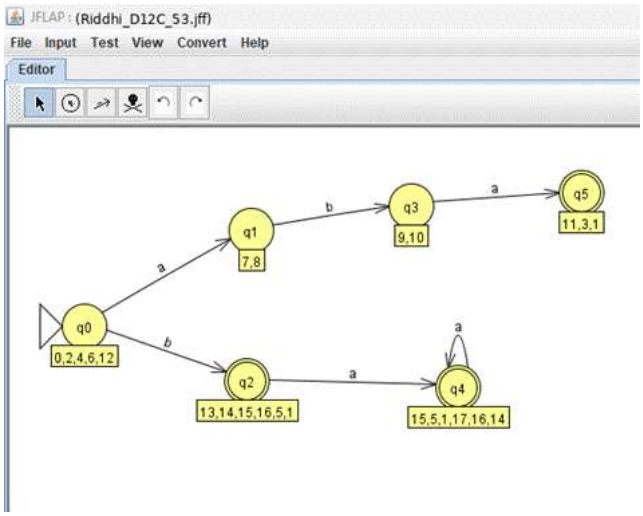
Step 8: And then upon connecting using epsilon transitions (similar to what we did in the previous
step), we get



Step 9: In the final step we need to break up a∗. Upon clicking the de-expressionify button you get two more states that are connected by an epsilon transition. To create the effect of a ∗ operation, remember that ∗ means 0 or more instances being concatenated with each other. So, create an epsilon transition loop from q18 to q19. And then connect up q18 to q20 and q21 to q19 with epsilon transitions. By doing this, the q20 to q21 transition can be used multiple times.



Step 10: The final step would be to convert this now to a DFA. The convert menu has a convert to the DFA option which can be used. Refer to the NFA to DFA conversion help to get more insight into this. In this example we will just use the 'Complete' button to do this in 1 step.



9) Mealy Machine

A Mealy machine is a finite automaton that **produces output**. Instead of accepting or rejecting, it writes an output symbol on every transition, so the output depends on the current state and the input symbol.

Formal Definition

A Mealy machine is a 6-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where:

- **Q** is a finite set of states
- **Σ** is the input alphabet
- **Δ** is the output alphabet
- **δ: Q × Σ → Q** is the transition function
- **λ: Q × Σ → Δ** is the output function
- **q_0** is the start state

There are no accepted states. The output has the same length as the input.

Example: detect "ab"

Over {a, b}, output 1 whenever the symbol just read completes "ab", and 0 otherwise. The states are A (last symbol was not a) and B (last symbol was a). Each cell shows next state / output:

State	a	b
A (start)	B / 0	A / 0
B	B / 0	A / 1

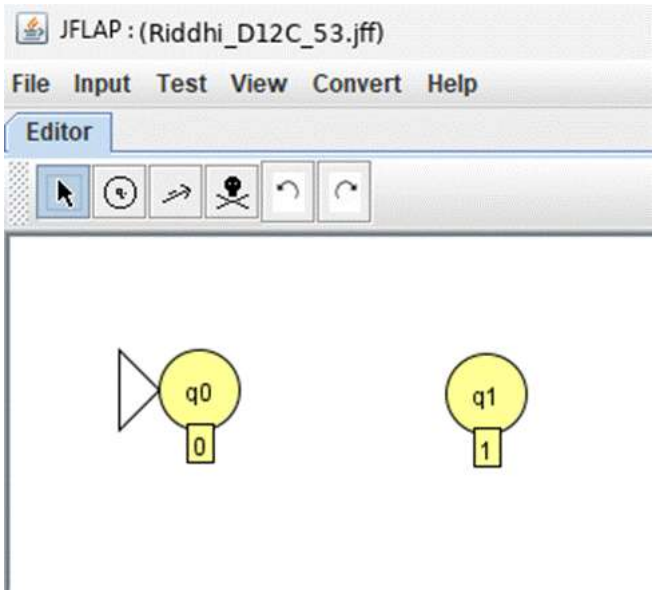
On input "aab": A —a/0→ B —a/0→ B —b/1→ A, so the output is **001**.

Example:

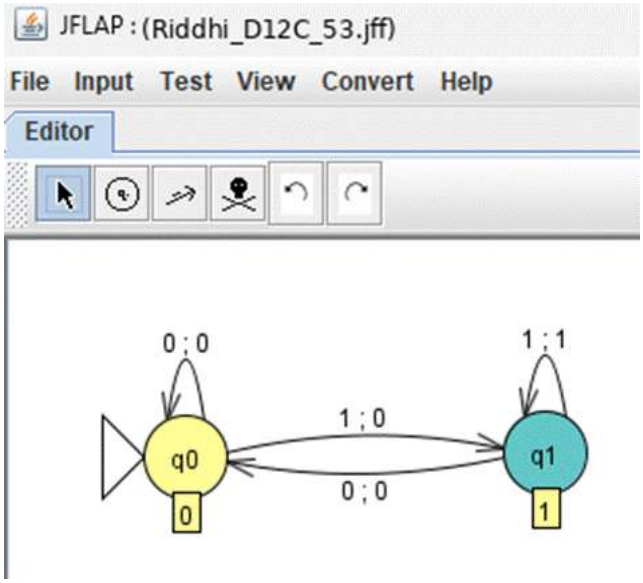
Construct a Mealy machine which takes a binary number and replaces the first 1 with a 0 from every substring starting with 1. For example, 0001001110 becomes 0000000110. This type of "bit stuffing" may be used in data transmission and telecommunications for run--length coding to

limit the number of consecutive bits of the same value. A bit of the opposite value is inserted after the maximum allowed number of consecutive bit

Step 1:
Create a Mealy machine with an initial state. Set its label to a “0” to remember that a zero is read. Next, add a second state to remember that a “1” was read. Label this with a “1



Step 2: Four transitions will be needed.
1. At q0, a 0 is read so a loopback to q0 is needed. Output a 0.
2. At q0, a 1 is read so a transition to q1 is needed. Output a 0 since this is the first 1 in a substring
starting with a 1.
3. At q1, a 0 is read so a transition to q0 is needed. Output a 0.
4. At q1, a 1 read so a loopback to q1 is needed. Output a 1



10) Moore Machine

A Moore machine is a finite automaton that **produces output**, where the output depends **only on the current state**. Each state carries an output symbol, which is written whenever the machine enters that state.

Formal Definition

A Moore machine is a 6-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where:

- **Q** is a finite set of states
- **Σ** is the input alphabet
- **Δ** is the output alphabet
- **δ: Q × Σ → Q** is the transition function
- **λ: Q → Δ** is the output function, defined on states
- **q_0** is the start state

There are no accepted states. The output has length |input| + 1, since the start state also produces an output before any symbol is read.

Example: detect "ab"

Over {a, b}, output 1 when the last two symbols read are "ab", and 0 otherwise. The states are A (last symbol was not a), B (last symbol was a), and C (just completed "ab"). Each state has an output, and each cell shows the next state:

State	Output	a	b
A (start)	0	B	A
B	0	B	C
C	1	B	A

On input "aab": A —a→ B —a→ B —b→ C, so the outputs are 0, 0, 0, 1, giving **0001**. The first 0 comes from the start state. Dropping it gives 001, the same as the Mealy machine.

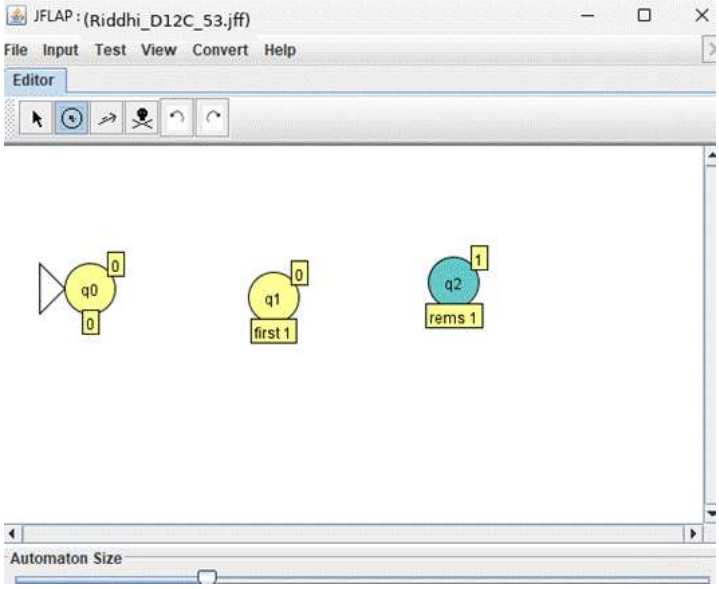
Example:

Construct a Moore machine which takes a binary number and replaces the first 1 with a 0 from every substring starting with 1. For example, 0001001110 becomes 0000000110. This type of "bit stuffing" may be used in data transmission and telecommunications for run--length coding to limit the number of consecutive bits of the same value. A bit of the opposite value is inserted after the maximum allowed number of consecutive bits.

Step 1:

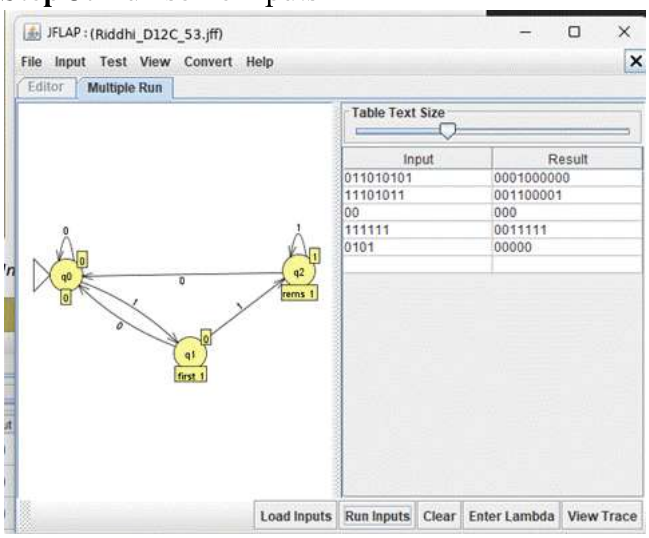
Create a Moore machine with an initial state. Create three states as follows:

1. q0 with an output of 0 to represent a 0 read.
2. q1 with an output of 0 to represent the first 1 read within a substring of 1s.
3. q2 with an output of 1 to represent the remaining 1's read after the first 1 was read.



Step 2:
Add the transitions:
1. At q0 and an input of 0, stay at q0.
2. At q0 and an input of 1, go to q1.
3. At q1 and an input of 0, go back to q0.
4. At q1 and an input of 1, go to q2.
5. At q2 and an input of 0, go all the way back to q0.
6. At q2 and an input of 1, stay at q2.

Step 3: Run some inputs



Conclusion

Thus, the experiments were successfully performed using JFLAP software to understand and implement different concepts of automata theory. The experiments demonstrated the construction and testing of Deterministic Finite Automata (DFA) and Non-Deterministic Finite Automata (NFA), conversion of NFA to DFA, DFA minimization, conversion of DFA into regular grammars and regular expressions, and the combination of automata to perform language operations. The conversion of regular expressions into finite automata was also explored. Additionally, Mealy and Moore machines were implemented to understand how finite-state machines generate outputs based on transitions and states, respectively. Overall, these experiments provided practical knowledge of finite-state machines, regular languages, and their applications using JFLAP.